Taylor & Francis
Taylor & Francis Group

International Journal of Production Research

ISSN: 0020-7543 (Print) 1366-588X (Online) Journal homepage: www.tandfonline.com/journals/tprs20

An exact algorithm for an identical parallel additive machine scheduling problem with multiple processing alternatives

Jun Kim & Hyun-Jung Kim

To cite this article: Jun Kim & Hyun-Jung Kim (2022) An exact algorithm for an identical parallel additive machine scheduling problem with multiple processing alternatives, International Journal of Production Research, 60:13, 4070-4089, DOI: 10.1080/00207543.2021.2007426

To link to this article: https://doi.org/10.1080/00207543.2021.2007426

INTERNATIONAL JOURNAL OF PRODUCTION RESEARCH
2022, VOL. 60, NO. 13, 4070–4089
https://doi.org/10.1080/00207543.2021.2007426

An exact algorithm for an identical parallel additive machine scheduling problem with multiple processing alternatives

Jun Kim[a] and Hyun-Jung Kim[b]

[a]Manufacturing Process Platform R&D Department, Korea Institute of Industrial Technology, Ansan, Republic of Korea; [b]Department of Industrial & Systems Engineering, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea

ABSTRACT

This paper develops an exact algorithm for the identical parallel additive machine scheduling problem by considering multiple processing alternatives to minimise the makespan. This research is motivated from an idea of elevating flexibility of a manufacturing system by using additive machines, such as 3D printers. It becomes possible to produce a job in a different form; a job can be printed in a complete form or in separate parts. This problem is defined as a bi-level optimisation model in which its upper level problem is to determine a proper processing alternative for each product, and its lower level problem is to assign the parts that should be produced to the additive machines. An exact algorithm, which consists of the linear programming relaxation of a one-dimensional cutting stock problem, a branch-and-price algorithm, and a rescheduling algorithm, is proposed to find an optimal solution of the problem. The experimental results show that the computational time of the algorithm outperforms a commercial solver (CPLEX). By examining how the parts are comprised when the processing alternatives are optimally selected, some useful insights are derived for designing processing alternatives of products.

ARTICLE HISTORY
Received 30 May 2021
Accepted 11 November 2021

KEYWORDS
Additive manufacturing system; parallel machine scheduling; multiple processing alternatives; minimization of makespan; exact algorithm

1. Introduction

Constructing intelligent manufacturing systems and applying them for mass customisation is a major objective of Industry 4.0. To achieve this, it is necessary to enhance the flexibility of manufacturing environments. Among the 10 types of manufacturing flexibility defined by Browne et al. (1984), five types are particularly relevant to the issue of mass customisation: (1) machine flexibility, (2) operation flexibility, (3) process flexibility, (4) product flexibility, and (5) production flexibility. As suggested by previous studies, the adaptation of additive manufacturing systems, such as three-dimensional (3D) printers, can be a solution to the five types of manufacturing flexibility (see Brettel, Klein, and Friederichsen 2016; Eyers et al. 2018; Delic and Eyers 2020). Various companies and institutions have already developed and used additive manufacturing systems to elevate manufacturing flexibility. Kang et al. (2018) proposed a factory as a service (FaaS) system that provides personalised production to support start-up companies by providing 3D printer-based manufacturing lines at a low price. A total of six FaaS systems are now being operated in South Korea. Automotive companies, such as BMW, Mercedes-Benz, and Ford, have also started to produce engine parts with 3D printers. Moreover, leading global companies, such as Siemens and SAP, are currently developing smart factory technologies and manufacturing testbeds with the help of 3D printers.

Although many studies have emphasised the necessity of additive manufacturing systems and the potential of these technologies (e.g. Bandyopadhyay and Bose 2015; Mai et al. 2016; Eyers and Potter 2017; Haleem and Javaid 2019), the strategies and techniques for operating the systems, such as production planning and scheduling with consideration for the system's characteristics, have been scarcely studied, as pointed out by Kim and Kim (2021). The first scheduling problem of a 3D printer-based manufacturing environment was proposed by Kim, Park, and Kim (2017) and Kim (2018). They focused on a characteristic of the 3D printer – its ability to produce different forms of products – and proposed a new scheduling problem by considering multiple processing alternatives in which a product can be produced in multiple ways. For example, as illustrated in Figure 1, product A has three processing alternatives when produced by a 3D printer: (1) it can be printed as a complete form of product A; (2) it can be printed separately with part *a*, part *b*, and part *c*; or (3) it can be printed separately

CONTACT Hyun-Jung Kim hyunjungkim@kaist.ac.kr Department of Industrial & Systems Engineering, Korea Advanced Institute of Science and Technology, 291 Daehak-ro, Daejeon 34141, Republic of Korea

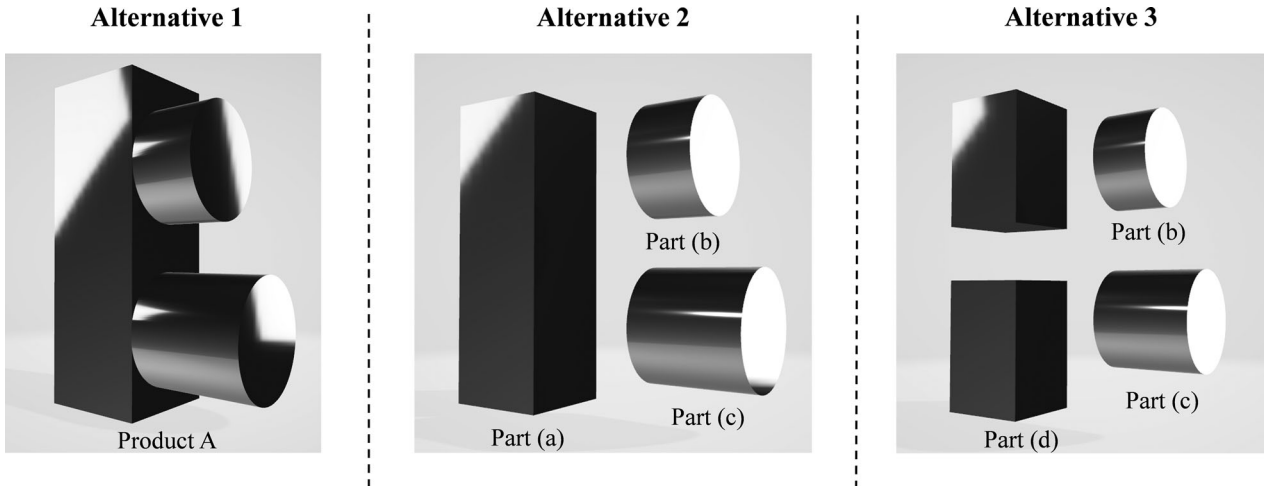

Figure 1. An example of three processing alternatives for producing a product.

with part *b*, part *c*, and two units of part *d*. The parts can be assembled later. The objective of the study was to determine which alternative should be selected for each product and to assign the parts to identical 3D printers in order to minimise the makespan (i.e. the maximum completion time). Kim and Kim (2021) showed that the makespan of a 3D printer-based additive manufacturing system is always less than or equal to that of a conventional manufacturing system that does not consider multiple processing alternatives. They proposed a genetic algorithm for the problem, and Kim (2018) analysed the worst case bound of list scheduling for the problem.

One limitation of the research conducted by Kim, Park, and Kim (2017), Kim (2018), and Kim and Kim (2021) is that the algorithms proposed to solve the problem do not guarantee optimality. Those studies have developed heuristic algorithms, such as a genetic algorithm or dispatching rules, and compared the solutions with lower bounds from which the gap is relatively large when the problem size is large. Therefore, this study proposes an exact algorithm for the parallel additive machine scheduling problem by considering multiple processing alternatives. There is no setup, and the objective is to minimise the makespan. The problem is defined and formulated as a bi-level optimisation problem in which the upper-level problem (ULP) is to select an optimal processing alternative for each product, and the lower-level problem (LLP) is to assign the realised parts to the additive machines. Then, a new exact algorithm, which solves the problem iteratively by considering the ULP and LLP until the optimal solution is derived, is proposed. In this research, we use the term 'realised parts' to indicate parts that are supposed to be produced as determined by the selection of processing alternatives for each product.

The contributions of this research can be summarised as follows: First, this study develops an optimal algorithm for the parallel additive manufacturing scheduling problem for the first time. Second, a bi-level mathematical model is proposed for the problem and solved efficiently by eliminating dominated solutions in the ULP with the proposed properties. Third, the rough optimality check with the LP relaxation is performed before deriving an exact solution and reduces its computational time (CT) significantly (refer to Table 8). Fourth, a rescheduling algorithm that modifies a feasible solution set of the ULP and provides a sequence of selected parts is proposed. Fifth, the proposed algorithm solves large-sized instances much faster than CPLEX (refer to Tables 2–4).

The rest of this paper is organised as follows. Section 2 reviews related works, and Section 3 defines the problem in detail with mathematical formulation. Section 4 proposes an exact algorithm composed of the linear programming (LP) relaxation of the dual problem of a parallel machine scheduling problem, a branch-and-price algorithm, and a rescheduling algorithm to derive the optimal solution to the problem. Section 5 experimentally evaluates the performance of the proposed algorithm in various instances. Finally, Section 6 concludes the paper and discusses future studies.

2. Literature review

This research addresses the scheduling problem of additive manufacturing systems composed of identical and parallel additive machines. The problem is similar to the classic parallel machine scheduling problem because after the processing alternatives of products are selected, the assignment of parts to machines becomes a parallel machine scheduling problem. Achieving an optimal

solution for the parallel machine scheduling problem is somewhat challenging, since solving even a simple problem involving only two machines with the objective of minimising makespan is NP hard, as proved by Karp (1972). Many studies have been conducted to solve the problem of parallel machine scheduling.

In the early days, studies have emphasised merely solving the problem and getting the optimal solution only for a small-sized problem. Since the late 1990s, there have been significant improvements in computational performance, which had led to more active research on finding the optimal solutions being conducted. Adopting the Dantzig–Wolfe decomposition method and the branch-and-bound algorithm, Chen and Powell (1999) developed an algorithm to solve the parallel machine scheduling problem by minimising the total weighted completion time. Mokotoff and Chrétienne (2002) proposed a cutting-plane algorithm for the unrelated parallel machine scheduling problem of minimising the makespan by identifying new valid inequalities in the convex hull. They showed that a problem with 20 machines and 200 jobs was solved in a computational time (CT) of 1098.51 s. Two years later, Mokotoff (2004) improved the previously developed algorithms by applying polyhedral theory and adding constraints regarding a lower bound. He showed that a parallel machine problem with 100 machines and 1000 jobs was exactly solved within a CT of 5098.40s. One year later, Dell'Amico and Martello (2005) insisted that their exact algorithms for the parallel machine scheduling problem could outperform Mokotoff (2004). Dell'Amico et al. (2008) then developed exact algorithms for the identical parallel machine scheduling problem–minimising the makespan–by combining a scatter search algorithm, a local search algorithm, and a branch-and-price algorithm. From this point on, research started to consider the more complex constraints of the parallel machine scheduling problem.

Kaplan and Rabadi (2012) considered an aerial refuelling scheduling problem that could be formulated as the parallel machine scheduling problem of minimising the total weighted tardiness. They considered additional constraints of ready times and a due-date-to-deadline window between the refuelling due date and a deadline to return without refuelling. They developed a dispatching rule and a simulated annealing-based heuristic algorithm, as well as an exact algorithm, to solve the problem. Ranjbar, Davari, and Leus (2012) assumed that there is some uncertainty as to the processing time of jobs and developed two branch-and-bound algorithms for finding an optimal solution to maximise the service level of customers. Hu, Ng, and Qin (2016) also considered the uncertainty of the parallel machine scheduling problem for plastic production. They assumed that the processing time and arrival time are uncertain but lie in their own intervals and also considered a sequence-dependent setup time. In order to derive a robust schedule, they developed an exact algorithm and an artificial bee colony algorithm. Kowalczyk and Leus (2017) extended the parallel scheduling problem of minimising the makespan to the problem of conflicting jobs. They developed an exact algorithm combining a branch-and-price algorithm with a bin-packing problem-based algorithm and a graph colouring method. Wu and Wang (2018) considered the additional constraint of multiprocessor tasks, that is, tasks that require more than one machine. They developed a branch-and-bound algorithm to obtain an optimal solution and also developed a tabu search-based algorithm to solve the large-scale problem within a rational CT. Fanjul-Peyro, Ruiz, and Perea (2019) reformulated a mixed-integer programming model of a parallel machine scheduling problem with sequence-dependent setup times and compared two exact algorithms provided by commercial solvers (CPLEX and GUROBI). Fanjul-Peyro (2020) proposed a mathematical model and an exact algorithm for an unrelated parallel machine scheduling problem with setups and resources. Kramer and Kramer (2021) addressed a discrete parallel machine scheduling location problem which combines facility location and job scheduling. They proposed an exact framework which combines an arc-flow formulation, a column generation algorithm, and three heuristics. Although a lot of research has sought to develop an exact algorithm for the parallel machine scheduling problem, no research has considered an exact algorithm for the parallel additive machine scheduling problem with multiple processing alternatives.

This problem we consider is similar to the scheduling problem with job splitting in that a job can be divided into various sub-jobs. The scheduling problem that considers a job splitting property was first proposed by Serafini (1996) with the practical need to schedule looms in a textile factory. He analysed some polynomially solvable cases with the due date-related measures. Since then, many studies on scheduling with job splitting have been conducted. Shim and Kim (2008) proposed a branch- and-bound algorithm to solve a parallel machine scheduling problem with a job splitting property for a printed circuit boards manufacturing system to minimise total tardiness. Park, Lee, and Kim (2012) examined a parallel machine scheduling problem with job splitting and sequence-dependent setup times. They developed three heuristic algorithms, slack-based, dynamic scheduling window-based, and estimated latest starting time-based algorithms, to minimise total tardiness of jobs, Fu, Aloulou, and Triki (2017) developed

a two-phase iterative heuristic algorithm to solve an unrelated parallel machine scheduling problem with job splitting and the vehicle routing problem with delivery time windows. Kim, Song, and Jeong (2020) proposed six simulated annealing and genetic algorithm-based metaheuristic algorithms combining two encoding schemes and three decoding methods to solve a parallel machine scheduling problem with job splitting to minimise total tardiness. Lee, Jang, and Kim (2021) addressed a parallel machine scheduling problem with job splitting and setup resource constraints while minimising makespan. They developed iterative job splitting algorithms to solve the problem and also derived a worst-case bound of the algorithms. Kim and Lee (2021) examined a uniform parallel machine scheduling problem by considering job splitting, sequence-dependent setup times, and capacitated servers. With the objective of minimising the makespan, they proposed a mathematical programming model and developed a heuristic algorithm. Job splitting is different from the proposed problem in that, in our problem, the parts are predefined from processing alternatives and are not able to be split into arbitrary forms. The job splitting property allows a job to be split into any sub-jobs. Therefore, only after a processing alternative is determined, a parallel machine scheduling problem can be solved in our problem.

The research gap between the previous studies and this work can be summarised as follows: Most previous studies on parallel machine scheduling do not assume that jobs can be split. Jobs in parallel machine scheduling with job splitting can be split into sub-jobs with any size, and there is no constraint on a specific size of separated sub-jobs. The parallel additive machine scheduling problem, which has recently been introduced and solved only with heuristics algorithms, provides a predefined form of parts in each processing alternative. To the best of our knowledge, this study is the first to develop an exact algorithm for the parallel machine scheduling problem with multiple processing alternatives.

3. Problem definition and formulation

3.1. Problem definition

The objective of the problem addressed in this research is to determine a processing alternative for each product and assign the parts from the chosen alternatives to machines while minimising the makespan of the additive manufacturing system. The system consists of M identical additive machines ($M \geq 2$), which can be processed in parallel. F products ($F \geq M$) are produced by the system, and each product has L printing alternatives. A set of parts to be produced is determined by selecting printing alternatives for each product. In this research, we assume that the lth printing alternative of a product is composed of l parts to be produced ($1 \leq l \leq L$). For example, when printing alternative 3 is chosen for the production of a product, the product is processed in three separate parts, which are assembled later. For the same product, the total processing time of parts for each processing alternative is assumed to be equal. There is no setup between parts, and the assembly time of parts is assumed to be negligible, so we only need to consider the assignment of parts to parallel additive machines. A machine cannot process more than a single part simultaneously, and pre-emption is not permitted.

The problem we consider contains two decision variables: (1) selection of processing alternatives and (2) assignment of parts to machines. These variables have a hierarchical relationship. Once processing alternatives for the products are selected, the parts to be produced can be determined and assigned to machines. Therefore, the problem can be presented as a bi-level optimisation problem, as illustrated in Figure 2. The ULP is defined as a problem that derives the first decision variable: the selection of processing alternatives. By solving the ULP, we can obtain a feasible set of processing alternatives for the product in Step 1 of Figure 2. Of course, the optimality of the candidate solutions cannot be guaranteed only by solving the ULP. One of the feasible solutions to the ULP is delivered to the LLP in Step 2 to derive the second decision variable: the assignment of parts to machines. The LLP problem is formulated in Step 3, and the LP relaxation of its dual problem is solved in Step 4. Then the weak optimality in Step 5 is checked, and if it passes the check, the dual problem of the LLP is solved to optimality by the branch-and-price algorithm in Step 6. Otherwise (i.e. presented as o/w in Step 6 of Figure 2), another feasible solution of the ULP is delivered to the LLP or LB (i.e. lower bound on the makespan for the ULP) is updated and a set of feasible solutions of the ULP is regenerated. When the optimal solution of the LLP is derived with the branch-and-price algorithm, the optimality of the solutions to the LLP and ULP is the confirmed by the strong optimality check in Step 7. If it is not optimal, a rescheduling algorithm is performed, and if it does not provide an optimal solution, the next iteration begins.

3.2. Mathematical formulation

Referring to Kim and Kim (2021), the parameters and decision variables used in the mathematical model are defined in Table 1.

Note that a_{fli} is a parameter that presents the structural relationship between parts, products, and processing alternatives. Based on these notations, we first introduce

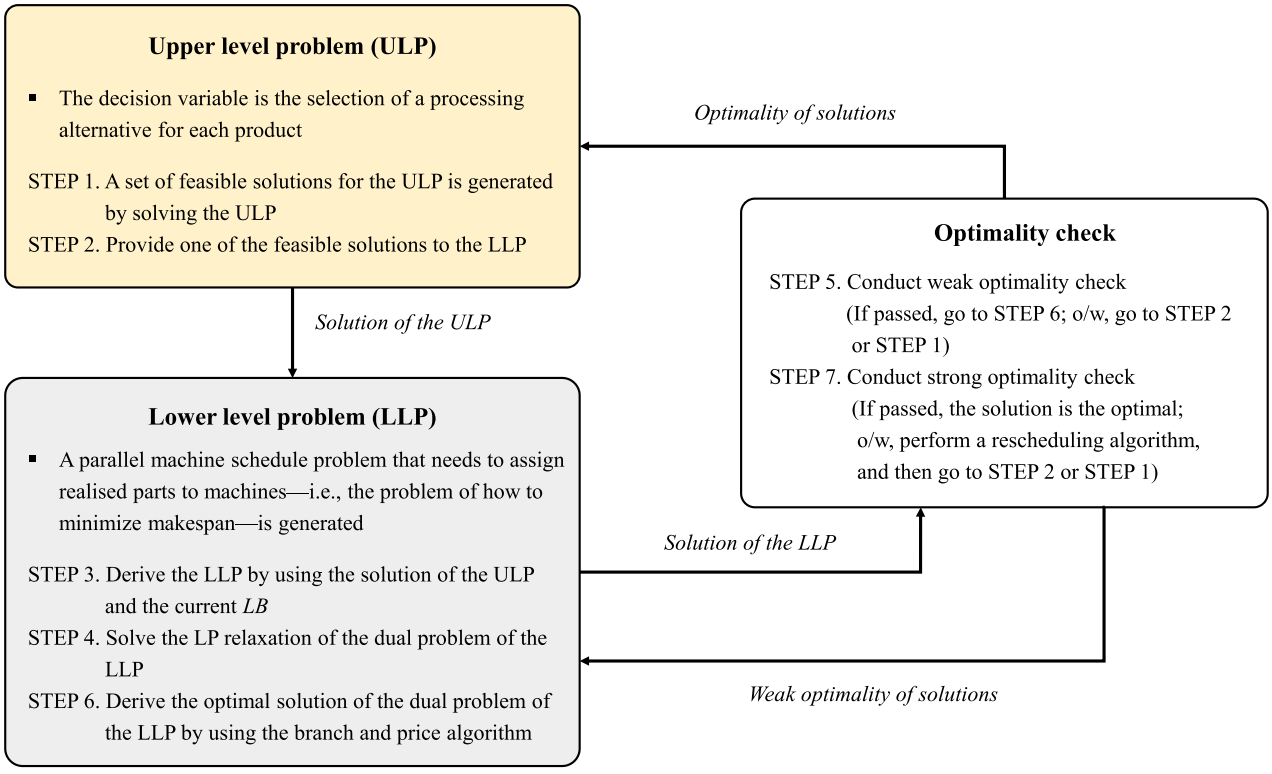

Figure 2. Structure of the proposed problem as a form of bi-level optimisation.

Table 1. Notations of the parameters and decision variables.

Parameters:	
i	index of a machine, $1 \le i \le M$
f	index of a product, $1 \le f \le F$
l	index of a processing alternative, $1 \le l \le L$
j	index of a part, $1 \le j \le N\,(=FL)$
a_{flj}	1 if part j is a component of alternative l of product f, and 0 otherwise
p_j	processing time of part j
Decision variables:	
x_{ij}	1 if part j is processed on machine i, and 0 otherwise
y_{fl}	1 if product f selects processing alternative l, and 0 otherwise
C_{max}	Makespan

the mathematical formulation of the problem as follows:

$$\min C_{max} \tag{1}$$

Subject to

$$\sum_{l} y_{fl} = 1 \; \forall f \tag{2}$$

$$\sum_{j} x_{ij} p_j \le C_{max} \; \forall i \tag{3}$$

$$\sum_{i} x_{ij} = \sum_{l} y_{fl} a_{flj} \; \forall f, j \tag{4}$$

$$x_{ij}, y_{fl} \in \{0,1\} \forall i, j, f \tag{5}$$

The objective of the problem, minimisation of the makespan, is presented in Equation (1). Equation (2) ensures that each product chooses only one processing alternative. Equation (3) makes the makespan equal to the largest completion time of machines, which are calculated as the summation of the total processing times of parts allocated to the machines. A part assigned to exactly one machine is represented by Equation (4), and binary decision variables x_{ij} and y_{fl} are described by Equation (5).

The mathematical model is then transformed into the bi-level mathematical formulation as follows:

$$\min_{y_{fl}} C_{max} \tag{6}$$

Subject to

$$\sum_{l} y_{fl} = 1 \; \forall f \tag{7}$$

$$\sum_{j} x_{ij} p_j \le C_{max} \forall i \tag{8}$$

$$x_{ij}, y_{fl} \in \{0,1\} \forall i, j, f \tag{9}$$

$$x_{ij} \in X, \tag{10}$$

where

$$X = \underset{x_{ij}}{\operatorname{argmin}}\, C_{max} \tag{11}$$

Subject to

$$\sum_i x_{ij} = \sum_l y_{fl} a_{flj} \forall f, j \tag{12}$$

The bi-level formulation consists of two separated parts (i.e. ULP and LLP). The objective of the ULP is to minimise the makespan regarding the decision variable y_{fl} as seen in Equation (6), while the LLP aims to find the decision variable x_{ij} that can minimise the makespan as presented in Equation (11). In other words, the purpose of the ULP is to find the optimal combination of selected processing alternatives, and the parts that are chosen from processing alternatives are optimally assigned to machines in the LLP. The relationship between the ULP and the LLP is described by Equations (10) and (11). Note that in Equation (11), $\text{argmin}_{x_{ij}} C_{max}$ means that we want to find the decision variable x_{ij} that minimises C_{max}. Equations (7), (8), (9), and (12) are equal with Equation (2), (3), (5), and (4) respectively.

4. Exact algorithm to solve the proposed problem

4.1. Structure of the exact algorithm

To better understand the proposed exact algorithm, we first briefly explain the structure of the algorithm. The exact algorithm consists of four stages (Figure 3). In the first stage, the initial value of *LB* is computed, and a set of feasible solutions for the ULP (which are combinations of the selected processing alternatives) is derived by using the proposed properties in Section 4.2.1. In the second stage, for each iteration, the optimality of the solution to the ULP is examined by the LP relaxation of the dual problem of the LLP (weak optimality check). The *LB* obtained in the first stage is used as a parameter in the LP relaxation. For the solutions to the ULP that pass the weak optimality check, the strong optimality is checked in the third stage. In the third stage, the optimal solution to the dual problem of the LLP is derived by using the branch-and-price algorithm; thus, the real optimality can be inspected. If the inspected solutions to the ULP are not found to be optimal, a rescheduling process is conducted in the fourth stage. If all of the feasible solutions to the ULP under the current *LB* are not optimal, the *LB* and the corresponding feasible solutions to the ULP are updated, and the four stages are repeated. In Figure 2, steps 1 and 2, steps 4 and 5, and steps 6 and 7 correspond to stages 1, 2, and 3 in Figure 3, respectively.

In the proposed algorithm, several solution techniques (i.e. the LP relaxation, branch-and-price algorithm, and rescheduling algorithm) are applied in each stage and work together to efficiently find an optimal solution. The LP relaxation in stage 2 and the branch-and-price algorithm in stage 3 are both used to check whether each feasible solution to the ULP derived in stage 1 is optimal by solving the LLP. As can be seen in Figure 3, the LP relaxation is first applied before using the branch-and-price algorithm because solving the LP relaxation of the LLP takes a much less CT even though the optimality is checked roughly. The branch-and-price algorithm is applied only for the solutions of the ULP verified with the LP relaxation, and this leads to the substantial CT reduction. A rescheduling algorithm is then used when all the solutions in stage 3 are revealed not to be optimal under the current *LB* of the makespan. There is a chance of finding an optimal solution from the rescheduling algorithm, and if an optimal solution is not found, a new iteration begins by updating the *LB*.

The convergence of the proposed exact algorithm to the optimal solution can be guaranteed by comparing the

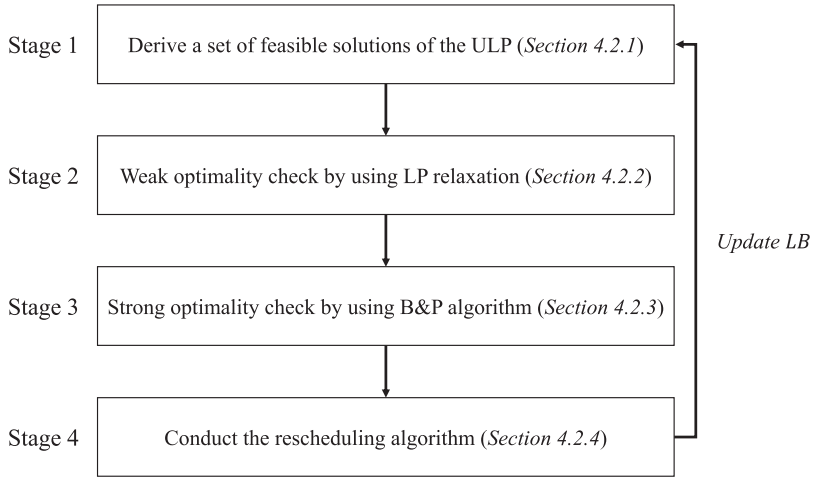

Figure 3. Four stages of the proposed algorithm.

makespan of the solution of the LLP given a feasible alternative set from the ULP and so far the best *LB*. From the fact that a solution is optimal when it is equal to the lower bound, the proposed algorithm has been designed to keep comparing the *LB* to the solution obtained and increase the *LB* by one when all of the feasible solutions of the ULP do not provide makespan which equals the current *LB*. In this manner, the proposed algorithm can find the best processing alternatives and detailed sequences of parts which have the makesan equal to the so far best *LB*, and this procedure guarantees the optimality.

4.2. Details of the exact algorithm

4.2.1. Deriving the lower bound and a feasible solution set of the ULP

The first stage of the proposed exact algorithm is to derive the initial *LB* of the problem. In the exact algorithm, the optimality of a pair of solutions (solutions to the related ULP and LLP) is analysed by using the best *LB* identified so far. Thus, it is important to set an initial *LB* to contain all possible feasible solutions and to update this when it is determined that the best *LB* identified so far is not, in fact, the optimal makespan. The initial *LB* is presented in Proposition 4.1.

Proposition 4.1: *The initial LB of the proposed algorithm is* $\lceil \sum_l \sum_f \sum_j p_j a_{flj}/ML \rceil$ *where* $\lceil k \rceil$ *is a smallest integer value larger than or equal to k.*

Proof: Since the *LB* is determined by solving the LLP (i.e. the parallel machine scheduling problem), the *LB* of the exact algorithm can be obtained from the parallel machine scheduling problem. The typical *LB* of the basic parallel machine scheduling problem is known as $\lceil \sum_j p_j/M \rceil$, which is the total processing time divided by the number of the machines (McNaughton 1959). According to the assumption of this problem, the total processing times for each processing alternative for the same product are equal. This indicates that no matter how each product's processing alternative is selected, the sum of the machine completion times is always the same as $\sum_l \sum_f \sum_j p_j a_{flj}/L$. Thus, for any LLP of the proposed problem, the *LB* can be computed with $\lceil \sum_l \sum_f \sum_j p_j a_{flj}/ML \rceil$. ■

After the *LB* is derived, the proposed exact algorithm generates a feasible solution set for the ULP, which includes candidates for the optimal solution. Whether a solution is included in the set or not is determined by checking that the LLP following the ULP's solution can be solved within the *LB*. The first property used in this stage is presented in Proposition 4.2.

Proposition 4.2: *Solutions that satisfy* $LB \geq \left(\max_{r \in R} p_r, p_{[M]} + p_{[M+1]} \right)$ *can be included in the feasible solution set of the ULP, where R is a set of realised parts and* $p_{[r]}$ *is the rth largest processing time among the realised parts.*

Proof: For the LLP problem, a modified lower bound on makespan can be obtained with $\max\left(\lceil \sum_r p_{r \in R}/M \rceil, \max_{r \in R} p_r, p_{[M]} + p_{[M+1]} \right)$ from Dell'Amico and Martello (1995). A solution to the ULP is optimal if and only if its *LB* is equal to the optimal makespan of the LLP. Since the optimal makespan of the LLP is always larger than or equal to the lower bound of the LLP, it is unnecessary to consider the solutions to the ULP in which the lower bound of its LLP is greater than *LB*. It is clear that $LB\, LB \geq \lceil \sum_r p_{r \in R}/M \rceil$, and therefore $LB \geq \left(\max_{r \in R} p_r, p_{[M]} + p_{[M+1]} \right)$ should be satisfied for the solutions to be included in the feasible solution set of the ULP. ■

We provide more properties for generating a feasible solution set for the ULP with the initial *LB*. When $\sum_l \sum_f \sum_j p_j a_{flj}/ML$, which is the initial *LB,* is an integer, if the completion times of all machines are equal to $\sum_l \sum_f \sum_j p_j a_{flj}/ML$, the solution is optimal for the given initial *LB*. To do that, the conditions in Propositions 4.3 and 4.4 have to be satisfied.

Proposition 4.3: *If* $\sum_l \sum_f \sum_j p_j a_{flj}/ML$ *is an even and integer value, the number of realised parts that have odd processing times should be* $2M(n-1)$*, where n is a certain positive integer value, to have the optimal makespan of* $\sum_l \sum_f \sum_j p_j a_{flj}/ML$.

Proof: If the optimal makespan is $\sum_l \sum_f \sum_j p_j a_{flj}/ML$ which is an even and integer value, the completion time of each machine must be $\sum_l \sum_f \sum_j p_j a_{flj}/ML$. In order to have the even number of completion times for each machine, the even number of parts that have odd processing times should be assigned to each machine. Thus, the number of realised parts that have odd processing time should always be an even and multiple of *M*. ■

Proposition 4.4: *If* $\sum_l \sum_f \sum_j p_j a_{flj}/ML$ *is an odd and integer value, the number of realised parts that have odd processing times should be* $M(2n-1)$*, where n is a positive integer value, to have the optimal makespan of* $\sum_l \sum_f \sum_j p_j a_{flj}/ML$.

Proof: If the optimal makespan is $\sum_l \sum_f \sum_j p_j a_{flj}/ML$, which is an odd and integer value, the completion time

of each machine must be $\sum_l \sum_f \sum_j p_j a_{fij}/ML$. In order to have the odd number of completion times for each machine, the odd number of parts that have odd processing times should be assigned to each machine. Thus, the number of realised parts that have odd processing times should always be an odd and multiple of M. ■

For the initial LB, the feasible solution set for the ULP is derived by applying Propositions 4.2–4.4. For the updated lower bound, Proposition 4.2 is only considered. Whenever all combinations of the processing alternatives for each product in the derived feasible solution set are found not to be optimal, the LB is updated to $LB + 1$.

4.2.2. Weak optimality check

Since it takes a relatively long time to find an optimal solution to the LLP generated from a feasible solution to the ULP, it is important to avoid examining solutions that cannot be optimal to reduce the number of iterations of the algorithm. In this stage, the solutions to the ULP are analysed by the LP relaxation of the dual problem of the LLP to filter out suboptimal solutions before the strong optimality check process is conducted.

Even though the solutions (selected processing alternatives) to the ULP are different, the corresponding LLPs can have the same composition of parts because different parts can have the same processing time, which leads to the same makespan. Therefore, those redundant solutions to the ULP are first eliminated. Next, the sequence of solutions to the ULP to be investigated is determined. The number of iterations and CT can be decreased significantly when an actual optimal solution is inspected earlier. In this work, each solution to the ULP is ranked by two criteria: (1) the standard deviation of the part processing times, and (2) the maximum processing time of the parts, which are revealed to be relatively small in an optimal solution, as illustrated by numerical experiments shown in Tables 12–14 in the Appendix. Solutions to the ULP are ranked from 1 in increasing order of the first criteria: the standard deviation of the part processing time. If the solutions have the same standard deviation, the solutions are ranked by the second criteria.

Each solution to the ULP is then inspected to determine whether it is the optimal solution or not with the one-dimensional cutting stock problem (1D-CSP), which is a well-known dual problem of the parallel machine scheduling problem. The 1D-CSP involves determining the smallest number of rolls with width W that are used for satisfying the demand of N clients with orders of b_j items of width w_j, where $j = 1, \ldots, N$. When we compare rolls, b_j, and w_j to machines, realised parts, and processing times, respectively, the 1D-CSP becomes similar to the LLP. Thus, when the number of minimised rolls is equal to the number of machines when W is set as the current LB, the optimality of the problem can be proved. Based on the Kantorovich model (1960), the transformed 1D-CSP as a suitable form for the LLP can be formulated as follows:

$$\min \sum_i z_i \tag{13}$$

Subject to

$$\sum_i x_{ij} \geq h_j \; \forall j \tag{14}$$

$$\sum_j p_j x_{ij} \leq LB z_i \; \forall i \tag{15}$$

$$z_i = 0 \text{ or } 1 \; \forall i \tag{16}$$

$$x_{ij} = 0 \text{ or } 1 \; \forall i, \forall j \tag{17}$$

where z_i and x_{ij} are the decision variables indicating whether machine i is used in the schedule or not and the assignment of realised part j to machine i respectively, and h_j, given from the ULP, indicates the realised parts; it is 1 if the alternative that is composed of part j is selected and 0 otherwise.

Before the optimality of solutions to the ULP is inspected with the optimal solution to the LLP, some of them can be filtered out by solving the LP relaxation of the problem. The LP relaxation of the problem is obtained by relaxing the integer variables to real-valued ones (i.e. Equations (16) and (17)). It then can be solved much faster than the original mixed integer programming model. The LP relaxation may not provide a feasible solution to the original LLP, but the solution can be used as the lower bound. Hence, if the lower bound is larger than the so far best LB, the corresponding solution is filtered out. Note that the so far best LB keeps increasing when no optimal solution is found. Therefore, the solution eliminated can be considered in the next iterations.

Since $\sum_i z_i$ indicates the minimum number of machines required to obtain the makespan equal to the current LB, when $\sum_i z_i$ is greater than M, the corresponding solution to the ULP cannot be the optimal solution; it thus does not need to be considered in the strong optimality check process. The CT of the algorithm can be reduced by conducting the weak optimality check prior to the strong optimality check because solving the LP relaxation problem requires a much less CT than solving the integer programming problem. In this research, we use the LP relaxation of the Dantzig–Wolfe decomposition model of the 1D-CSP introduced by Vance (1998) for the weak optimality check which is known to be provide better solution than the simple 1D-CSP. The LP relaxation model of the decomposed LLP can be formulated

as follows:

$$\min \sum_i \sum_u \lambda_i^u \tag{18}$$

Subject to

$$\sum_i \sum_u x_{ij}^u \lambda_i^u \geq h_j \quad \forall j \tag{19}$$

$$\sum_u \lambda_i^u \leq 1 \; \forall i \tag{20}$$

$$\sum_u x_{ij}^u \lambda_i^u \geq 0 \; \forall i, j \tag{21}$$

$$\lambda_i^u \geq 0 \; \forall i, \; u \tag{22}$$

The knapsack constraint for machine i, regarding Equation (14), is substituted by a convex combination of the extreme points (i.e. a solution set of the knapsack constraint of machine i which is a polytope), $(x_{i1}^u, x_{i2}^u, \ldots, x_{iN}^u)^T, u = 1, \ldots, U$ of $conv\{x_{ij} : \sum_j p_j x_{ij} \leq LB, x_{ij} \geq 0$ and integer $\forall j\}$ in the Dantzig-Wolfe decomposition model of the 1D-CSP. Here, u denotes an index of the valid cutting patterns (feasible set of parts that can be assigned to a machine in our problem) of the given width of a roll (LB in our problem). In the model, a new decision variable λ_i^u is used which indicates whether pattern u is used in machine i. The objective of the decomposed model is to minimise cutting patterns used as Equation (18). Equation (15) of the original 1-D CSP is transformed to Equation (19), and Equations (20)–(22) ensure the cutting pattern for each machine's designated completion time be a feasible solution. For more details on the Dantzig-Wolfe decomposition model, see Vance (1998).

4.2.3. Strong optimality check

After a solution to the ULP passes the weak optimality check described in Section 4.2.2, the final optimality of the solution is inspected, and the optimal solution to the LLP is derived. In this research, the optimal solution to the LLP is obtained by using a branch-and-price algorithm of the 1D-CSP. When the LLP is defined as a form of 1D-CSP by using the current LB and realised parts, the branch-and-price algorithm modified from Vance (1998) is conducted to obtain an optimal solution to the 1D-CSP.

In the branch-and-price algorithm, the Dantzig-Wolfe decomposition model of the LLP becomes a restricted-master-problem, and a problem for solving the knapsack constraint to derive valid cutting patterns is a sub-problem. The structure of the branch-and-price algorithm modified from Vance (1998) is shown in Figure 4. The lower and upper bounds are updated whenever column generation is cut off and an integer solution

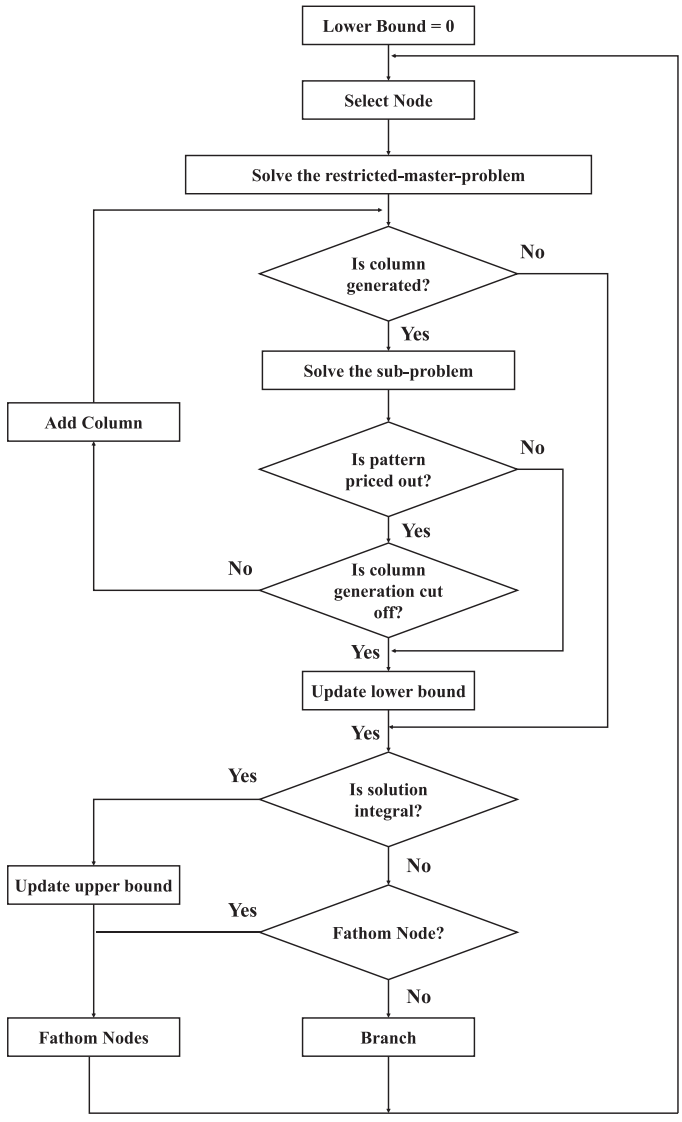

Figure 4. Flowchart of the branch-and-price algorithm (Vance 1998).

is derived, respectively, and the lower bound on the optimal solution of the 1D-CSP is used to prevent unnecessary column generation. The lower bound is updated by rounding up the largest LP relaxation solution and compared with the original lower bound and upper bound (i.e. so far best integer solution) for pricing out the patterns and fathoming nodes. When the updated lower bound is smaller than that of the original, a pattern derived by solving the sub-problem is priced out. A node is also fathomed when the lower bound is greater than the upper bound. Decisions on generating and cutting off columns are made by comparing the current solution (i.e. LP bound) updated by solving the restricted-master-problem's LP relaxation and the sub-problem with the lower bound. When the LP bound is larger than the lower bound, columns are not generated anymore and cut off.

For more details on the branch-and-price algorithm, see Vance (1998).

When the objective value of the optimal solution, $\sum_i z_i$, is equal to M, the corresponding combination of processing alternatives and machine assignment become the optimal solution to the ULP and LLP, respectively, and the proposed algorithm can be terminated. If $\sum_i z_i$ of the optimal solution to the 1D-CSP is greater than M, the LLP cannot be solved within the current lower bound on the makespan; therefore, the corresponding solution to the ULP is determined not to be optimal. Then, the rescheduling algorithm (described in Section 4.2.4) is conducted for the schedule obtained as the optimal solution to the LLP.

4.2.4. Rescheduling algorithm

Before considering another feasible solution to the ULP, a rescheduling algorithm is conducted, as presented in Figure 5. When the makespan of a revised schedule from the algorithm is equal to the current *LB*, the corresponding solutions to the ULP and LLP become optimal. Otherwise, the optimality of another feasible solution to the ULP is investigated through the weak and strong optimality check procedures.

The rescheduling algorithm can be summarised as follows: First, a given schedule is modified by reassigning parts that were assigned to machines whose indexes are larger than M to the machines with the indexes less than or equal to M with the longest processing time (LPT) rule. It then tries to reassign parts on machines with large completion times to the machines with small completion times. If there exists a product whose parts are all processed on the same machine, machine k, and the product has a part with the processing time of $C_k - LB$ in another processing alternative, the corresponding alternative is selected and the part is reassigned to the machine having the smallest completion time (machine q). This procedure makes the completion time of all machines equal to *LB*. If a new schedule having the makspan equal to *LB* is obtained, it is optimal.

4.2.5. Proposed exact algorithm

The proposed exact algorithm, including the details of the algorithm described in Sections 4.2.1–4.2.4, is presented in Figure 6. In the algorithm, the *LB* is first computed with Proposition 4.1, and a set of solutions of the ULP, denoted as *S*, are generated by Propositions 4.2–4.4 depending on the *LB*. Then the solutions in *S* are ordered by considering the realised parts' processing times, and each solution, from the one most likely to be optimal, is investigated with the LP relaxation and the branch-and-price algorithm. Then the rescheduling algorithm

```
For a schedule obtained by applying a branch-and-price algorithm to 1D-CSP, reassign parts that are allocated to machines whose index i is larger than M to the machines with indexes less than or equal to M according to the LPT rule;
Set A = {i|C_i > LB};
Set B = {i|C_i < LB};
While |A| > 0 and |B| > 0,
    Order machine indexes of A in decreasing order of C_i and B in increasing order of C_i;
    Let k and q denote indexes of machines in A and B respectively, and k ← 1, q ← 1;
    If there exists a product in which its all parts are assigned to machine k,
        if the product has a part whose processing time is C_k − LB in another processing alternative, L̄,
            change the product's processing alternative to L̄ and assign the part having the processing time of C_k − LB to machine q and other parts to machine k;
            Update set A and B;
        else
            end the algorithm;
    else
        end the algorithm;
End the algorithm;
```

Figure 5. The proposed rescheduling algorithm.

$LB \leftarrow \lceil \sum_l \sum_f \sum_j p_j a_{flj} / ML \rceil$;
$C_{max} \leftarrow V$ (Large number);
If $\sum_l \sum_f \sum_j p_j a_{flj} / ML$ is an integer and even number,
    generate set S that includes solutions of the ULP by considering Propositions 2 and 3;
else if $\sum_l \sum_f \sum_j p_j a_{flj} / ML$ is an integer and odd number,
    generate set S that includes solutions of the ULP by considering Propositions 2 and 4;
else
    generate set S that includes solutions of the ULP by considering Proposition 2;
While $C_{max} > LB$,
    Remove some redundant solutions in S;
    Order ULP solution indexes in S in increasing order of the rank by considering standard deviation and the maximum value of realised parts' processing times;
    Let s denote an index of a solution in S, and $s \leftarrow 1$;
    While $s \leq |S|$,
        Generate a LPP corresponding to solution s in a form of 1D-CSP;
        Solve the LP-relaxation of the Dantzig decomposition formulation of the 1D-CSP;
        If $\sum_i z_i > M$,
            $s \leftarrow s + 1$;
        else
            conduct the branch-and-price algorithm to the 1D-CSP;
            If $\sum_i z_i = M$,
                the optimal solution is found, and $C_{max} \leftarrow LB$;
            else
                conduct the rescheduling algorithm;
                If |A| = 0 and |B| = 0 after the rescheduling algorithm is completed,
                    the optimal solution is found, and $C_{max} \leftarrow LB$;
                else
                    $s \leftarrow s + 1$;
    $LB \leftarrow LB + 1$;
    Update S by considering Proposition 2;
End the algorithm;

Figure 6. The proposed exact algorithm.

is applied if it is not optimal, and a whole procedure is repeated if no optimal solution is found even after the rescheduling algorithm.

5. Numerical experiments

We now conduct the experiments to show the efficiency of the proposed exact algorithm. The CT and the makespan of the algorithm are compared to those from CPLEX for small-sized, medium-sized, and large-sized instances in Tables 2–4 to verify the superiority of the proposed algorithm. Note that CPLEX was only able to provide an optimal solution for small-sized instances within 10 hours. We then perform experiments to show the effectiveness of considering multiple processing alternatives by comparing the makespan to that having

Table 2. Experimental results of small-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		CT – EA (s)	CT – CPLEX (s)	RG (%)	CT – EA (s)	CT – CPLEX (s)	RG (%)	CT – EA (s)	CT – CPLEX (s)	RG (%)
$M = 2$	$F = 3$	5.51	1.11	0	3.19	9.18	0	2.28	67.79	0
	$F = 5$	12.84	68.23	0	9.82	201.76	0	6.43	987.42	0
	$F = 10$	32.47	2201.27	0	41.91	4997.55	0	55.90	8711.26	0
	$F = 15$	40.19	4267.69	0	72.42	7804.47	0	179.53	13174.55	0
Average		22.75	1634.57	0	31.83	3253.24	0	61.03	5735.25	0
$M = 3$	$F = 3$	7.72	2.97	0	5.62	15.71	0	2.17	80.43	0
	$F = 5$	12.62	80.70	0	11.77	257.19	0	5.47	1311.08	0
	$F = 10$	35.41	2707.48	0	41.97	5641.72	0	82.73	10279.15	0
	$F = 15$	43.29	6318.54	0	70.23	11286.68	0	296.65	23865.40	0
Average		24.76	2277.42	0	32.39	4300.32	0	96.75	8884.01	0

Table 3. Experimental results of medium-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		CT – EA (s)	CT – CPLEX (s)	RG (%)	CT – EA (s)	CT – CPLEX (s)	RG (%)	CT – EA (s)	CT – CPLEX (s)	RG (%)
$M = 5$	$F = 20$	61.63	36,000	0.96	188.53	36,000	1.33	987.67	36,000	1.86
	$F = 30$	183.58	36,000	1.19	753.97	36,000	1.35	1625.19	36,000	1.41
	$F = 50$	1227.34	36,000	1.48	1751.61	36,000	1.78	2516.37	36,000	1.89
	$F = 100$	2089.22	36,000	1.54	2618.16	36,000	1.90	3541.11	36,000	2.07
Average		890.44	36,000	1.29	1328.07	36,000	1.59	2167.59	36,000	1.81
$M = 10$	$F = 20$	66.41	36,000	1.24	190.16	36,000	1.79	988.44	36,000	1.90
	$F = 30$	190.13	36,000	1.64	759.65	36,000	1.60	1667.53	36,000	2.01
	$F = 50$	1225.71	36,000	1.87	1813.37	36,000	2.19	2523.49	36,000	2.86
	$F = 100$	2101.73	36,000	2.62	2644.29	36,000	2.73	3608.75	36,000	2.98
Average		895.99	36,000	1.84	1351.87	36,000	2.07	2197.05	36,000	2.43

Table 4. Experimental results of large-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		CT – EA (s)	CT – CPLEX (s)	RG (%)	CT – EA (s)	CT – CPLEX (s)	RG (%)	CT – EA (s)	CT – CPLEX (s)	RG (%)
$M = 15$	$F = 100$	2818.48	36,000	4.36	3015.77	36,000	5.64	4287.46	36,000	5.89
	$F = 150$	4055.44	36,000	9.79	5133.48	36,000	10.12	6843.11	36,000	10.87
	$F = 200$	5817.52	36,000	13.57	6717.35	36,000	15.73	10352.43	36,000	16.07
Average		4230.48	36,000	9.24	4955.53	36,000	10.50	7161	7161	10.94
$M = 20$	$F = 100$	3208.21	36,000	5.08	3210.61	36,000	5.99	4621.17	36,000	6.83
	$F = 150$	4472.39	36,000	11.79	5527.68	36,000	12.82	7905.16	36,000	13.77
	$F = 200$	601.17	36,000	14.64	7521.08	36,000	16.10	11221.55	36,000	16.98
Average		2760.59	2760.59	10.50	5419.79	36,000	11.64	7915.96	7915.96	12.53

only one processing alternative for small-sized, medium-sized, and large-sized instances in Tables 5–7. Finally, the reduction rate of the CT obtained by considering Propositions, the rank-based inspection sequence, the weak optimality check process, and the rescheduling algorithm is presented in Tables 8–10. Each numerical experiment was iterated 50 times on an Intel(R) Core(TM) i9-10900KF CPU @ 3.70 GHz personal computer with 32 GB memory.

5.1. Computational time and makespan comparison

We first conduct numerical experiments to show the CT of the proposed exact algorithm and relative gap in the makespan derived by the proposed algorithm and CPLEX with a time limit. The experiments are carried out with three groups of instances: small-sized, medium-sized, and large-sized instances. The instances which can be optimally solved by CPLEX within 10 hours are considered to be small-sized ones, and the instances where CPLEX cannot provide an optimal solution within 10 hours but can provide a feasible one having the less than 3% gap from the lower bound are used as medium-sized ones. The remaining instances which have the gap larger than or equal to 3% in CPLEX within 10 hours are used as large-sized ones. Note that the samples were generated by following the criteria in Kim and Kim (2021). In the small-sized instances, M is set to 2 and 3, and F is set to 3, 5, 10, and 15. For the medium-sized instances, M is set to 5 and 10, and F is set to 20, 30, 50, and 100. In the large-size instances, M is set to 15 and 20, and F is set to

Table 5. Comparison results of small-sized instances.

		$L = 1$ vs $L = 2$		$L = 1$ vs $L = 3$		$L = 1$ vs $L = 5$	
		IR (%)	*p*-value	IR (%)	*p*-value	IR (%)	*p*-value
$M = 2$	$F = 3$	34.87	0.00057	39.02	0.00184	38.94	0.00079
	$F = 5$	30.43	0.00300	35.69	0.00110	36.46	0.00047
	$F = 10$	12.46	0.00519	16.93	0.00532	16.34	0.00426
	$F = 15$	7.09	0.00779	9.33	0.00582	8.96	0.00684
Average		21.21	0.0041375	25.24	0.00352	25.18	0.00309
$M = 3$	$F = 3$	38.73	0.00082	42.29	0.00068	43.01	0.00036
	$F = 5$	31.18	0.00125	36.04	0.00096	36.11	0.00115
	$F = 10$	14.31	0.00446	18.50	0.00525	17.92	0.00395
	$F = 15$	8.75	0.00796	10.56	0.00661	10.76	0.00507
Average		23.24	0.0036225	26.85	0.003375	26.95	0.0026325

Table 6. Comparison results of medium-sized instances.

		$L = 1$ vs $L = 2$		$L = 1$ vs $L = 3$		$L = 1$ vs $L = 5$	
		IR (%)	*p*-value	IR (%)	*p*-value	IR (%)	*p*-value
$M = 5$	$F = 20$	6.97	0.00854	8.01	0.00623	7.97	0.00781
	$F = 30$	4.15	0.00969	5.56	0.00886	5.75	0.00837
	$F = 50$	2.10	0.01021	2.88	0.01073	3.05	0.00981
	$F = 100$	1.48	0.01903	2.01	0.01608	1.99	0.01812
Average		3.68	0.0118675	4.62	0.010475	4.69	0.0110275
$M = 10$	$F = 20$	7.02	0.00781	8.32	0.00547	7.74	0.00630
	$F = 30$	4.54	0.00801	5.25	0.00718	5.48	0.00652
	$F = 50$	2.27	0.00970	2.94	0.00918	3.20	0.00887
	$F = 100$	1.23	0.01120	1.85	0.00990	1.69	0.01011
Average		3.77	0.00918	4.59	0.0079325	4.53	0.00795

Table 7. Comparison results of large-sized instances.

		$L = 1$ vs $L = 2$		$L = 1$ vs $L = 3$		$L = 1$ vs $L = 5$	
		IR (%)	*p*-value	IR (%)	*p*-value	IR (%)	*p*-value
$M = 15$	$F = 100$	2.13	0.00902	2.52	0.00957	2.63	0.01040
	$F = 150$	1.07	0.01240	1.48	0.01344	1.38	0.01918
	$F = 200$	0.68	0.03103	0.93	0.02344	0.85	0.03094
Average		1.29	0.017483333	1.64	0.015483333	1.62	0.020173333
$M = 20$	$F = 100$	2.27	0.00892	2.94	0.00905	2.83	0.00941
	$F = 150$	1.40	0.01388	1.79	0.02020	1.63	0.01964
	$F = 200$	1.02	0.02909	1.37	0.02019	1.45	0.02538
Average		1.56	0.017296667	2.03	0.01648	1.97	0.018143333

Table 8. Reduction rate of the CT for the small-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		W.O.C & RS (%)	Prop (%)	R.I (%)	W.O.C & RS (%)	Prop (%)	R.I (%)	W.O.C & RS (%)	Prop (%)	R.I (%)
$M = 2$	$F = 3$	9.82	10.53	2.82	8.89	10.46	3.49	6.93	11.75	3.57
	$F = 5$	12.80	10.56	3.46	13.31	10.08	5.19	12.11	10.64	5.83
	$F = 10$	16.87	11.79	5.55	17.63	11.12	5.76	16.78	10.95	5.67
	$F = 15$	18.72	10.20	5.10	19.12	10.13	7.18	18.86	11.25	6.74
Average		14.55	10.77	4.23	14.74	10.45	5.41	13.67	11.15	5.45
$M = 3$	$F = 3$	7.52	10.45	2.21	8.04	10.83	2.98	7.84	10.43	2.51
	$F = 5$	13.28	11.73	4.43	14.64	11.96	4.38	14.26	10.06	5.59
	$F = 10$	17.89	11.64	4.05	16.93	11.79	6.20	15.14	11.82	7.99
	$F = 15$	16.93	10.38	6.59	17.38	10.57	5.96	16.85	10.61	7.61
Average		11.58	11.05	4.32	14.25	11.29	4.88	13.52	10.73	5.93

∗ **W.O.C & RS**: Weak optimality check and rescheduling algorithm; **Prop**: Propositions; **R.I**: Rank-based inspection.

100, 150, and 200. L is considered to be 2, 3, and 5. The processing times for alternative 1 for each product are randomly generated between 100 and 500 min. Then the processing time of each part is randomly generated while maintaining the total processing time. Note that the CT solved using the proposed exact algorithm and CPLEX are presented for the small-sized instances, and only the CT of the proposed exact algorithm are presented for the medium-sized and large-sized instances, because the CPLEX cannot provide an optimal solution within

Table 9. Reduction rate of the CT for the medium-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		W.O.C & RS (%)	Prop (%)	R.I (%)	W.O.C & RS (%)	Prop (%)	R.I (%)	W.O.C & RS (%)	Prop (%)	R.I (%)
$M = 5$	$F = 20$	16.21	11.07	9.16	18.67	10.26	10.18	19.55	11.09	11.44
	$F = 30$	16.69	11.35	10.69	19.75	11.72	11.96	18.12	10.48	10.95
	$F = 50$	19.04	10.39	14.74	21.48	11.96	15.75	20.62	12.43	13.19
	$F = 100$	23.13	12.09	13.81	23.97	12.18	16.42	23.54	10.62	12.42
Average		18.77	11.22	12.10	20.97	11.53	13.58	20.46	11.16	12.00
$M = 10$	$F = 20$	15.12	11.75	10.57	17.21	10.66	12.40	17.33	12.47	13.90
	$F = 30$	19.89	10.46	12.99	21.27	11.16	14.94	20.73	11.83	13.37
	$F = 50$	23.71	12.38	12.28	25.06	12.09	14.24	26.18	10.41	14.72
	$F = 100$	25.54	11.23	13.47	26.64	10.43	15.25	26.40	12.48	15.57
Average		21.07	11.46	12.33	22.55	11.09	14.21	22.66	11.80	14.39

Table 10. Reduction rate of the CT for the large-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		W.O.C & RS (%)	Prop (%)	R.I (%)	W.O.C & RS (%)	Prop (%)	R.I (%)	W.O.C & RS (%)	Prop (%)	R.I (%)
$M = 15$	$F = 100$	26.78	10.69	14.83	28.38	10.88	18.79	28.49	10.27	19.50
	$F = 150$	29.60	11.51	16.18	31.02	11.70	22.32	30.84	11.00	25.04
	$F = 200$	31.41	11.84	21.31	32.74	12.24	25.82	31.16	10.84	23.95
Average		29.26	11.35	17.44	30.71	11.61	22.31	30.16	10.70	22.83
$M = 20$	$F = 100$	28.62	10.08	14.04	31.80	10.88	19.80	30.75	11.75	20.83
	$F = 150$	32.68	11.70	20.95	32.88	10.52	22.15	33.05	11.80	24.35
	$F = 200$	32.52	12.24	23.41	32.95	12.80	27.08	32.58	11.36	27.25
Average		31.27	11.34	19.47	32.54	11.40	23.01	32.13	11.64	24.14

10 hours for medium-sized and large-sized instances. The gap in the makespan solved by the proposed exact algorithm and CPLEX is calculated as Equation (23) where C_{max}^{CPLEX} and C_{max}^{EA} denote the makespan derived by CPLEX and the exact algorithm respectively. For the medium- and large-sized instances that cannot guarantee the optimal solution within 10 hours, we have derived C_{max}^{CPLEX} by constraining the CT as 10 hours.

$$\text{Relative Gap (RG)} = \frac{C_{max}^{CPLEX} - C_{max}^{EA}}{C_{max}^{EA}} \times 100\ (\%) \quad (23)$$

Table 2 shows the experimental results of small-sized instances. The average CT of the proposed algorithm is 44.98 s, with the minimum and maximum CT of 2.28 and 296.65 s, respectively. In terms of the CT, although the CPLEX outperforms the proposed exact algorithm in the extremely small instances of ($M = 2, F = 3, L = 2$) and ($M = 3, F = 3, L = 2$), the algorithm outperforms all the other instances. Comparing the CT of the proposed algorithm with CPLEX in which the CT dramatically increases by 23865.4 s as the number of machines, products, and processing alternatives increase, it is observed that the problem can be optimally solved within a rational CT using the proposed exact algorithm. As can be seen in Table 2, the CT of the proposed exact algorithm (CT-EA) decreases as the number of processing alternatives considered increases when ($M = 2, F = 3$), ($M = 2, F = 5$), ($M = 3, F = 3$), and ($M = 3, F = 5$). This is due to the small number of parts that would be produced. When multiple processing alternatives are considered, the makespan decreases because the parts with small processing times help balance the machine workloads. Therefore, when there are insufficient parts in a solution, the effectiveness of machine balancing decreases, and the gap between the optimal makespan and the initial lower bound can be large. This situation causes more iterations of the proposed algorithm, which causes the CT to increase accordingly.

As presented in Table 2, we have checked that the makespans from our algorithm and from CPLEX are equal for all of the 1200 small-sized instances. The fact that two methods provide the same makespan for each instance verifies that the proposed exact algorithm guarantees the optimal solution. In order to show that the proposed exact algorithm outperforms the CPLEX in terms of the CT, we have conducted the paired t-test, and the results are presented in Table 11 in the Appendix.

Table 3 shows the performance of the proposed exact algorithm for medium-sized instances. The average CT of the algorithm is 1471.84 s, with the minimum and maximum CT of 61.63 and 3608.75 s, respectively. The CT increases as the number of machines, products, and processing alternatives increases. The average relative gap is 1.84%, with the minimum value of 0.96% when ($M = 5, F = 20, L = 2$) and the maximum value of 2.98% when ($M = 10, F = 100, L = 5$). The relative gap is observed to increase as the number of machines, products, and considered processing alternatives increases.

When the problem size increases, the number of possible sequences considered to derive the optimal solution increases accordingly. However, the solution obtained within 10 hours from CPLEX is compared, and hence, the relative gap keeps increasing along with the size of the problem.

Table 4 presents the performance of the proposed algorithm with large-sized instances. The minimum CT, maximum CT, and average CT are 2818.48, 11221.55, and 5407.23 s, respectively. The average relative gap of the makespan is 10.89%. Similarly, the relative gap increases as the problem size increases. From the experimental results in Tables 2–4, we can claim that the proposed exact algorithm outperforms the CPLEX.

5.2. Comparison with the problem having one processing alternative

The experiments are conducted to show the effectiveness of considering multiple processing alternatives. The makespan of the problem having one processing alternative is used for the comparison. The improvement rate of the makespan for each instance in Section 5.1 is calculated as Equation (24) where C_{max}^{single} and $C_{max}^{multiple}$ denote the makespan of an instance having single processing alternative and multiple processing alternatives, respectively. In addition, the paired t-test is conducted to demonstrate the difference between two values in statistical aspects. The average improvement rate and p-values of small-sized, medium-sized, and large-sized instances are presented in Tables 5–7.

$$\text{Improvement Rate (IR)} = \frac{C_{max}^{single} - C_{max}^{multiple}}{C_{max}^{multiple}} \times 100\ (\%) \tag{24}$$

Table 5 shows the comparison results of the small-sized instances in which the average improvement rate is 24.78%, but it dramatically decreases to 7.09% as the number of products increases. It means that there is a limit on the makespan improvement even with the multiple processing alternatives. In other words, the difference between makespans is similar regardless of the number of products but the makespan itself increases as more products are considered. Therefore, the improvement rate decreases when the number of products increases. The experimental results also show that the improvement rate is larger when $(L = 3)$ or $(L = 5)$ compared to $(L = 2)$ while they are similar when $(L = 3)$ and $(L = 5)$. It indicates that considering more than two processing alternatives can improve the makespan, but it is not necessary to have too many processing alternatives. All the p-values are observed to be smaller than 0.05, so it is certain that multiple processing alternatives can improve the makespan.

Tables 6 and 7 present the comparison results of medium-sized and large-sized instances, respectively. As the number of products increases, the improvement rate of the makespan is observed to be decreased similarly with the small-sized instances. The average improvement rate is 4.31% and 1.69% for medium-sized and large-sized instances, respectively.

5.3. Reduction of computational times by components of the proposed exact algorithm

In the proposed exact algorithm, the several properties are applied to reduce the CT, such as the generation of a feasible solution set for the ULP by using Propositions 4.2–4.4, rank-based optimality inspection sequence of solutions to the ULP, weak optimality check, and application of the rescheduling algorithm. In this part of the numerical experiments, the extent to which the CT is reduced by these components for small-, medium-, and large-sized instances is examined. In detail, three cases are addressed. First, we examine a reduction rate of CT derived when using the weak optimality check and the rescheduling algorithm. When these two components are eliminated, the LLP becomes equal to the general parallel machine scheduling problem solved by the branch-and-price algorithm. The comparison results can indicate how the proposed components are superior than the branch-and-price algorithm that previous studies have used. Second, the reduction rate of CT derived when Propositions 4.2–4.4 are considered is examined. The feasible solutions of the ULP are determined by the propositions. The results can demonstrate the efficiency of the propositions filtering out the non-optimal solutions. Third, we examine how much CT reduces when the rank-based inspection is considered. It shows how the sequence of inspecting feasible solutions of the ULP affects the CT.

Table 8 shows the reduction rate of the CT for the small-sized instances. The contribution of the weak optimality check and the rescheduling algorithm to the reduction of the CT increases as the number of products increases. However, there is no clear increasing or decreasing tendency according to the number of processing alternatives. The average reduction rate of the small-sized instances is 13.88% with the minimum of 6.93% when $(M = 2, F = 3, L = 5)$ and the maximum of 19.12% when $(M = 2, F = 15, L = 3)$. Propositions 4.2–4.4 reduce the CT by 10.91% on average. The contribution of the propositions is observed to be similar regardless of the instance sizes. When the rank-based inspection sequence is applied, the CT is reduced by 5.04% on average. Note that the application of the

rank-based inspection sequence is more effective when the problem is large.

Tables 9 and 10 present the reduction rate of the CT for the medium- and large-sized instances, respectively. They show similar features as those in Table 8. The weak optimality check and rescheduling algorithm reduce the CT by 21.08% and 31.01% for the medium- and large-sized instances, respectively. Propositions 4.2–4.4 are revealed to reduce the CT by 11.37% and 11.34% on average for the medium- and large sized instances. The reduction rate of the rank-based inspection on the medium- and large sized-instances is observed to be 13.10%, and 21.53%, respectively. Note that the weak optimality check and rescheduling algorithm contribute to the CT reduction the most, and Propositions 4.2–4.4 affects the reduction rate of the CT as the second best.

6. Conclusion

In this research, we considered the identical parallel additive machine scheduling problem with multiple processing alternatives to achieve the high flexibility of manufacturing systems. We defined the problem with a bi-level optimisation model in which the ULP is to select processing alternatives for products and the LLP is to assign the realised parts to additive machines. We then developed an exact algorithm composed of four stages; In the first stage, a set of the feasible solutions for the ULP is generated by using the proposed properties; In the second stage, the optimality of each solution of the ULP is investigated by solving the LP relaxation of the dual problem of the LLP; Then, an optimal solution is searched with the branch-and-price algorithm; In the final stage, if a solution is revealed not to be optimal, the rescheduling algorithm is carried out. The four stages are repeated until the optimal solution of the problem is obtained by updating the lower bound on makespan consistently.

In the experiments, the performance of the proposed algorithm has been first compared to CPLEX in terms of the CT for small-sized instances. The algorithm was able to solve medium-sized instances with 10 machines and 100 products and large-sized instances with 20 machines and 200 products to optimality within 3608.75 and 11221.55 s, respectively, whereas CPLEX was not provide an optimal solution within 10 hours. We have further conducted experiments to show the effectiveness of considering multiple processing alternatives and showed that the makespan was improved by 10.26% on average. We then have examined how much the components of the proposed exact algorithm contribute to reducing the CT. In the experiments, the weak optimality check and rescheduling algorithm, Propositions 4.2–4.4, and the rank-based inspection have reduced the CT by 21.99%, 11.21%, and 13.22% on average.

We finally provide some practical insights, useful for designing part compositions in the processing alternatives, which can be derived from the numerical experiments conducted in Section 5. As illustrated in Section 5.2, the degree of makespan reduction has been much larger when ($L = 3$) or ($L = 5$) is considered than ($L = 2$). Also, the makespan has been reduced by 9.13%, 10.83%, and 10.82% by considering multiple processing alternatives when ($L = 2$), ($L = 3$), and ($L = 5$), respectively. Therefore, it would be better to consider at least three processing alternatives when designing components of processing alternatives. The instances of ($L = 3$) and ($L = 5$) do not have distinct differences on the makespan improvement. Since the CT increases when the number of processing alternatives increases, generating three processing alternatives for each product is recommended. We have also identified that the standard deviations of processing times in optimal solutions decrease as the number of considered processing alternatives increases (See Tables 12–14 in the Appendix). The standard deviations also increase as more products are produced. Therefore, the system productivity can be enhanced by designing the processing alternatives to have parts in a wider range of processing times as the problem size and the number of alternatives considered decrease.

In the future, this work can be extended by considering more complex requirements, such as sequence-dependent setup times, unrelated additive machines, assembly lines, ready times, and due dates. Different objective functions can also be considered, such as minimising the total tardiness or service level of the customer.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This paper was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (Ministry of Science Industry and Technology) (No. 2019R1C1C1004667).

Notes on contributors

Jun Kim is a researcher in the Digital Transformation R&D Department, Korea Institute of Industrial Technology. He is currently pursuing his Ph.D. degree from the Sungkyunkwan University (South Korea) and received his Bachelor's degree from the same university in 2014. His research interest is optimisation, scheduling, smart factory, digital transformation, and data analytics.

Hyun-Jung Kim received the Ph.D. degree in Industrial and Systems Engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, Republic of Korea in 2013. She was a Postdoctoral Researcher in the Department of Industrial Engineering & Operations Research, University of California, Berkeley, and was an Assistant Professor in the Department of Systems Management Engineering, Sungkyunkwan University, Republic of Korea. She is now an Assistant Professor with the Department of Industrial & Systems Engineering, KAIST. Her research interests include modelling, scheduling, and control of production systems.

Data availability statement

The data that supports the findings of this study is available from the first author, Jun Kim, upon reasonable request.

References

Bandyopadhyay, A., and A. Bose. 2015. *Additive Manufacturing*. New York: CRC Press.

Brettel, M., M. Klein, and N. Friederichsen. 2016. "The Relevance of Manufacturing Flexibility in the Context of Industrie 4.0." *Procedia CIRP* 41: 105–110.

Browne, J., D. Dubois, K. Rathmill, S. P. Sethi, and K. E. Stecke. 1984. "Classification of Flexible Manufacturing Systems." *The FMS Magazine* 2 (2): 114–117.

Chen, Z. L. 1999. "Solving Parallel Machine Scheduling Problems by Column Generation." *INFORMS Journal on Computing* 11 (1): 78–94.

Delic, M., and D. R. Eyers. 2020. "The Effect of Additive Manufacturing Adoption on Supply Chain Flexibility and Performance: An Empirical Analysis from the Automotive Industry." *International Journal of Production Economics* 228: 107689.

Dell'amico, M., M. Iori, S. Martello, and M. Monaci. 2008. "Heuristic and Exact Algorithms for the Identical Parallel Machine Scheduling Problem." *INFORMS Journal on Computing* 20 (3): 333–344.

Dell'Amico, M., and S. Martello. 1995. "Optimal Scheduling of Tasks on Identical Parallel Processors." *ORSA Journal on Computing* 7 (2): 191–200.

Dell'Amico, M., and S. Martello. 2005. "A Note on Exact Algorithms for the Identical Parallel Machine Scheduling Problem." *European Journal of Operational Research* 160 (2): 576–578.

Eyers, D. R., and A. T. Potter. 2017. "Industrial Additive Manufacturing: A Manufacturing Systems Perspective." *Computers in Industry* 92: 208–218.

Eyers, D. R., A. T. Potter, J. Gosling, and M. M. Naim. 2018. "The Flexibility of Industrial Additive Manufacturing Systems." *International Journal of Operations & Production Management* 38 (12): 2313–2343.

Fanjul-Peyro, L. 2020. "Models and an Exact Method for the Unrelated Parallel Machine Scheduling Problem with Setups and Resources." *Expert Systems with Applications: X* 5: 100022.

Fanjul-Peyro, L., R. Ruiz, and F. Perea. 2019. "Reformulations and an Exact Algorithm for Unrelated Parallel Machine Scheduling Problems with Setup Times." *Computers & Operations Research* 101: 173–182.

Fu, L. L., M. A. Aloulou, and C. Triki. 2017. "Integrated Production Scheduling and Vehicle Routing Problem with Job Splitting and Delivery Time Windows." *International Journal of Production Research* 55 (20): 5942–5957.

Haleem, A., and M. Javaid. 2019. "Additive Manufacturing Applications in Industry 4.0: A Review." *Journal of Industrial Integration and Management* 4 (4): 1930001.

Hu, H., K. K. H. Ng, and Y. Qin. 2016. "Robust Parallel Machine Scheduling Problem with Uncertainties and Sequence-Dependent Setup Time." *Scientific Programming* 2016: 5127253.

Kang, H. S., S. Do Noh, J. Y. Son, H. Kim, J. H. Park, and J. Y. Lee. 2018. "The FaaS System Using Additive Manufacturing for Personalized Production." *Rapid Prototyping Journal* 24 (9): 1486–1499.

Kantorovich, L. V. 1960. "Mathematical Methods of Organizing and Planning Production." *Management Science* 6 (4): 366–422.

Kaplan, S., and G. Rabadi. 2012. "Exact and Heuristic Algorithms for the Aerial Refueling Parallel Machine Scheduling Problem with Due Date-to-Deadline Window and Ready Times." *Computers and Industrial Engineering* 62 (1): 276–285.

Karp, R. M.. 1972. "Reducibility among Combinatorial Problems" In *Complexity of Computer Computations*, edited by R. E. Miller, J. W. Thatcher, and J. D. Bohlinger, 85âĂŞ103. Boston: Springer.

Kim, H. J. 2018. "Bounds for Parallel Machine Scheduling with Predefined Parts of Jobs and Setup Times." *Annals of Operations Research* 261: 401–412.

Kim, J., and H. J. Kim. 2021. "Parallel Machine Scheduling with Multiple Processing Alternatives and Sequence-Dependent Setup Times." *International Journal of Production Research* 59 (18): 5438–5453.

Kim, H. J., and J. H. Lee. 2021. "Scheduling Uniform Parallel Dedicated Machines with Job Splitting, Sequence-Dependent Setup Times, and Multiple Servers." *Computers & Operations Research* 126: 105115.

Kim, J., S. S. Park, and H. J. Kim. 2017. "Scheduling 3D Printers with Multiple Printing Alternatives." In *2017 13th IEEE Conference on Automation Science and Engineering (CASE)*, 488–493.

Kim, J. G., S. Song, and B. Jeong. 2020. "Minimising Total Tardiness for the Identical Parallel Machine Scheduling Problem with Splitting Jobs and Sequence-Dependent Setup Times." *International Journal of Production Research* 58 (6): 1628–1643.

Kowalczyk, D., and R. Leus. 2017. "An Exact Algorithm for Parallel Machine Scheduling with Conflicts." *Journal of Scheduling* 20 (4): 355–372.

Kramer, R., and A. Kramer. 2021. "An Exact Framework for the Discrete Parallel Machine Scheduling Location Problem." *Computers & Operations Research* 132: 105318.

Lee, J. H., H. Jang, and H. J. Kim. 2021. "Iterative Job Splitting Algorithms for Parallel Machine Scheduling with Job Splitting and Setup Resource Constraints." *Journal of the Operational Research Society* 72 (4): 780–799.

Mai, J., L. Zhang, F. Tao, and L. Ren. 2016. "Customized Production Based on Distributed 3D Printing Services in Cloud Manufacturing." *The International Journal of Advanced Manufacturing Technology* 84: 71–83.
McNaughton, R. 1959. "Scheduling with Deadlines and Loss Functions." *Management Science* 6 (1): 1–12.
Mokotoff, E. 2004. "An Exact Algorithm for the Identical Parallel Machine Scheduling Problem." *European Journal of Operational Research* 152 (3): 758–769.
Mokotoff, E., and P. Chrétienne. 2002. "A Cutting Plane Algorithm for the Unrelated Parallel Machine Scheduling Problem." *European Journal of Operational Research* 141 (3): 515–525.
Park, T., T. Lee, and C. O. Kim. 2012. "Due-Date Scheduling on Parallel Machines with Job Splitting and Sequence-Dependent Major/Minor Setup Times." *The International Journal of Advanced Manufacturing Technology* 59 (1): 325–333.
Ranjbar, M., M. Davari, and R. Leus. 2012. "Two Branch-and-Bound Algorithms for the Robust Parallel Machine Scheduling Problem." *Computers & Operations Research* 39 (7): 1652–1660.
Serafini, P. 1996. "Scheduling Jobs on Several Machines with the Job Splitting Property." *Operations Research* 44 (4): 617–628.
Shim, S. O., and Y. D. Kim. 2008. "A Branch and Bound Algorithm for an Identical Parallel Machine Scheduling Problem with a Job Splitting Property." *Computers & Operations Research* 35 (3): 863–875.
Vance, P. H. 1998. "Branch-and-Price Algorithms for the One-Dimensional Cutting Stock Problem." *Computational Optimization and Applications* 9 (3): 211–228.
Wu, L., and S. Wang. 2018. "Exact and Heuristic Methods to Solve the Parallel Machine Scheduling Problem with Multi-Processor Tasks." *International Journal of Production Economics* 201: 26–40.

Appendix

Paired *t*-test to verify that the proposed exact algorithm outperforms the CPLEX

For the small-sized instances in which both the proposed exact algorithm and CPLEX provide an optimal solution, we conducted the paired *t*-test to verify that the exact algorithm outperforms the CPLEX in terms of the CT statistically. Table 11 shows the *p*-value of each size of instances. Except for the smallest instance with ($M = 2$, $F = 3$, and $L = 2$), all the other instances have the *p*-value less than 0.05. In other words, it is demonstrated that the proposed exact algorithm outperforms the CPLEX regarding the CT.

Statistical values of the processing time of small-sized instances

Numerical experiments are conducted to find the characteristics of the composition of parts realised by the optimal processing alternative for each product. The instances are the same as in the first part of the experiment (see Section 5.1). We calculate the standard deviation of the part processing times and the maximum processing time divided by the average processing time of the parts when the solution is optimal.

Table A1. Results of the paired *t*-test for the small-sized instances.

		$L = 2$	$L = 3$	$L = 5$
$M = 2$	$F = 3$	0.99815	0.00681	0.00513
	$F = 5$	0.00571	0.00424	0.00368
	$F = 10$	0.00148	0.00112	0.00120
	$F = 15$	0.00089	0.00071	0.00068
$M = 3$	$F = 3$	0.00781	0.00522	0.00361
	$F = 5$	0.00512	0.00207	0.00218
	$F = 10$	0.00103	0.00091	0.00080
	$F = 15$	0.00075	0.00056	0.00021

Table 12 shows the results of the experiments for small-sized instances. As can be seen in Table 12, the standard deviations of processing times decrease as the number of considered processing alternatives increases. The standard deviations also increase as more products are produced. Although the maximum processing time divided by the average processing time decreases when multiple processing alternatives are considered, there are no differences between ($L = 2$), ($L = 3$), and ($L = 5$).

Tables 13 and 14 present the experimental results of medium-sized and large-sized instances, respectively. Similarly, the standard deviations in processing times decrease when multiple processing alternatives are considered; however, the standard deviations are fairly similar regarding the changes in L and F, unlike in the small-sized instances. The maximum processing time divided by the average processing time are observed to have the same tendency as those found in the small-sized instances.

The results presented in Tables 12–14 can be used when the part composition of the processing alternatives is designed. Moreover, the results indicate that when feasible solutions to the ULP are inspected, the CT can be reduced by searching solutions with a low standard deviation of realised part processing times first.

Comparison with the benchmark problem

Additional experiments are conducted to compare the performance of the proposed exact algorithm and the heuristic algorithm developed by Kim and Kim (2021). They developed a travelling salesman problem-based genetic algorithm to solve a parallel machine scheduling problem with multiple processing alternatives and sequence-dependent setup times. Difference between the problem and this study comes from whether considering the sequence-dependent setup time. In order to compare the performance of the two algorithms, we set sequence-dependent setup times to 0 and derived solutions for the same instances used in Section 5.1. The improvement rate of the makespan for each instance is calculated as Equation (25) where C_{max}^{GA} and C_{max}^{EA} denote the makespan from the genetic algorithm and the exact algorithm respectively. The average values and CT of small-sized, medium-sized, and large-sized instances are presented in Table 15–17.

$$\text{Improvement Rate (IR)} = \frac{C_{max}^{GA} - C_{max}^{EA}}{C_{max}^{exactalgorithm}} \times 100\ (\%) \quad (25)$$

Table 15 shows the comparison results of the exact algorithm and the genetic algorithm for small-sized instances. As can be seen in Table 15, the improvement rate of the exact algorithm for extremely small-sized instances is 0 since the genetic

Table A2. Statistical values of the processing time of small-sized instances.

		$L = 1$		$L = 2$		$L = 3$		$L = 5$	
		Std	Max/average	Std	Max/average	Std	Max/average	Std	Max/average
$M = 2$	$F = 3$	45.77	2.71	32.95	2.63	24.68	2.62	19.61	2.53
	$F = 5$	41.17	2.60	31.73	2.33	20.70	2.58	17.93	2.60
	$F = 10$	37.05	2.64	24.13	2.52	19.95	2.52	17.37	2.41
	$F = 15$	34.82	2.82	22.09	2.37	16.51	2.35	15.55	2.33
Average		39.70	2.69	27.73	2.46	20.46	2.52	17.62	2.47
$M = 3$	$F = 3$	43.29	2.71	37.82	2.55	29.73	2.44	20.83	2.37
	$F = 5$	40.04	2.78	34.05	2.41	23.89	2.53	18.06	2.54
	$F = 10$	32.43	2.62	28.89	2.54	21.33	2.47	15.06	2.60
	$F = 15$	31.18	2.79	24.97	2.48	18.87	2.38	13.24	2.36
Average		36.74	2.73	31.43	2.50	23.46	2.46	16.80	2.47

∗Std: Standard deviation.

Table A3. Statistical values of the processing time of medium-sized instances.

		$L = 1$		$L = 2$		$L = 3$		$L = 5$	
		Std	Max/average	Std	Max/average	Std	Max/average	Std	Max/average
$M = 5$	$F = 20$	30.90	2.82	10.34	2.56	11.84	2.39	10.61	2.53
	$F = 30$	23.56	2.79	15.20	2.48	17.13	2.61	15.27	2.52
	$F = 50$	22.74	2.61	14.09	2.51	15.28	2.41	14.40	2.47
	$F = 100$	22.13	2.72	11.31	2.65	12.69	2.34	12.41	2.39
Average		24.83	2.74	12.74	2.55	14.24	2.44	13.17	2.48
$M = 10$	$F = 20$	27.79	2.67	14.65	2.54	10.52	2.47	14.64	2.41
	$F = 30$	29.66	2.71	10.12	2.67	11.39	2.51	13.75	2.35
	$F = 50$	21.84	2.75	11.17	2.32	15.72	2.63	13.88	2.52
	$F = 100$	24.06	2.63	13.84	2.53	13.17	2.30	11.95	2.38
Average		25.84	2.69	12.45	2.52	12.70	2.48	13.56	2.42

Table A4. Statistical values of the processing time of large-sized instances.

		$L = 1$		$L = 2$		$L = 3$		$L = 5$	
		Std	Max/average	Std	Max/average	Std	Max/average	Std	Max/average
$M = 15$	$F = 100$	23.45	2.83	11.50	2.41	10.68	2.57	13.36	2.62
	$F = 150$	25.14	2.71	12.02	2.65	14.57	2.48	12.24	2.33
	$F = 200$	22.80	2.76	11.59	2.32	9.44	2.39	10.66	2.51
Average		23.80	2.77	11.70	2.46	11.56	2.48	12.09	2.49
$M = 20$	$F = 100$	22.19	2.81	13.33	2.52	10.99	2.40	13.12	2.51
	$F = 150$	24.18	2.75	10.39	2.59	11.22	2.52	11.43	2.57
	$F = 200$	22.54	2.69	10.53	2.44	8.72	2.36	9.87	2.42
Average		22.97	2.75	11.42	2.52	10.31	2.43	11.47	2.50

Table A5. Comparison results of small-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		CT – EA (s)	CT – GA (s)	IR (%)	CT – EA (s)	CT – GA(s)	IR (%)	CT – EA (s)	CT – GA (s)	IR (%)
$M = 5$	$F = 20$	5.51	0.22	0	3.19	0.21	0	2.28	0.25	0
	$F = 30$	12.84	0.23	0	9.82	0.25	0	6.43	0.28	0
	$F = 50$	32.47	0.31	0.52	41.91	0.29	0	55.90	0.41	0.81
	$F = 100$	40.19	0.35	0.97	72.42	0.35	1.25	179.53	0.44	1.83
Average		22.75	0.28	0.37	31.83	0.228	0.31	61.03	0.35	0.66
$M = 3$	$F = 3$	7.72	0.24	0	5.62	0.24	0	2.17	0.32	0
	$F = 5$	12.62	0.27	0	11.77	0.29	0	5.47	0.37	0.8
	$F = 10$	35.41	0.30	0.59	41.97	0.37	0	82.73	0.43	0
	$F = 15$	43.29	0.36	1.22	70.23	0.41	1.76	296.65	0.69	1.24
Average		24.76	0.29	0.45	32.39	0.33	0.44	96.75	0.45	0.51

algorithm also provides the optimal solution. For the small-sized instances, the average improvement rate of makespan is 0.46% on average, while the CT of the proposed algorithm is larger than that of the genetic algorithm.

Tables 16 and 17 present the comparison results of medium-sized and large-sized instances, respectively. As the size of instances increases, the improvement rate of the makespan tends to increase. The average improvement rate is 3.24% and 6.17% for medium-sized and large-sized instances respectively. CT of the exact algorithm is much longer than the genetic algorithm. The results presented in Tables 15–17 indicate that the proposed exact algorithm outperforms the genetic algorithm for all the three sizes of instances, although it takes longer time to derive solutions.

Table A6. Comparison results of medium-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		CT – EA (s)	CT – GA (s)	IR (%)	CT – EA (s)	CT – GA (s)	IR (%)	CT – EA (s)	CT – GA (s)	IR (%)
$M = 5$	$F = 20$	61.63	4.27	3.55	188.53	5.13	3.64	987.67	7.64	2.71
	$F = 30$	183.58	4.78	3.87	753.97	6.25	3.47	1625.19	10.23	2.97
	$F = 50$	1227.34	6.58	2.97	1751.61	8.27	3.21	2516.37	23.87	3.48
	$F = 100$	2089.22	11.56	3.19	2618.16	14.26	3.37	3541.11	29.91	3.09
Average		890.44	6.80	3.40	1328.07	8.48	3.42	2167.59	17.91	3.06
$M = 10$	$F = 20$	66.41	7.15	3.23	190.16	7.69	3.80	988.44	15.51	3.10
	$F = 30$	190.13	8.21	3.19	759.65	13.49	2.84	1667.53	25.72	3.06
	$F = 50$	1225.71	15.44	3.42	1813.37	19.50	3.18	2523.49	31.08	3.41
	$F = 100$	2101.73	23.58	2.60	2644.29	28.75	3.13	3608.75	42.11	3.39
Average		895.99	13.60	3.11	1351.87	17.36	3.24	2197.05	28.61	3.24

Table A7. Comparison results of large-sized instances.

		$L = 2$			$L = 3$			$L = 5$		
		CT – EA (s)	CT – GA (s)	IR (%)	CT – EA (s)	CT – GA (s)	IR (%)	CT – EA (s)	CT – GA (s)	IR (%)
$M = 15$	$F = 100$	2818.48	41.74	6.40	3015.77	57.42	6.42	4287.46	72.70	6.85
	$F = 150$	4055.44	241.57	5.16	5133.48	285.16	6.25	6843.11	305.44	5.32
	$F = 200$	5817.52	387.11	5.39	6717.35	400.06	6.27	10352.43	431.72	6.04
Average		4230.48	223.47	5.65	4955.53	247.55	6.31	7161	269.95	6.07
$M = 20$	$F = 100$	3208.21	68.24	5.58	3210.61	91.54	5.73	4621.17	102.48	6.99
	$F = 150$	4472.39	301.25	6.74	5527.68	410.21	7.39	7905.16	450.17	6.32
	$F = 200$	601.17	412.47	7.14	7521.08	480.71	6.10	11221.55	631.20	5.04
Average		2760.59	260.65	6.49	5419.79	327.49	6.41	7915.96	394.62	6.12